OpenPilot: A Pipeline That Drives

Conceptual Architecture of an Open-Source Driver Assistance System

Group F:
Sachin Kumar · Hardik Arora · Ahmed Elnahass · Minseo Kim · Mark Susanto

EECS 4314 – Advanced Software Engineering
York University · Assignment 1 · 22/10/2026

YORK U



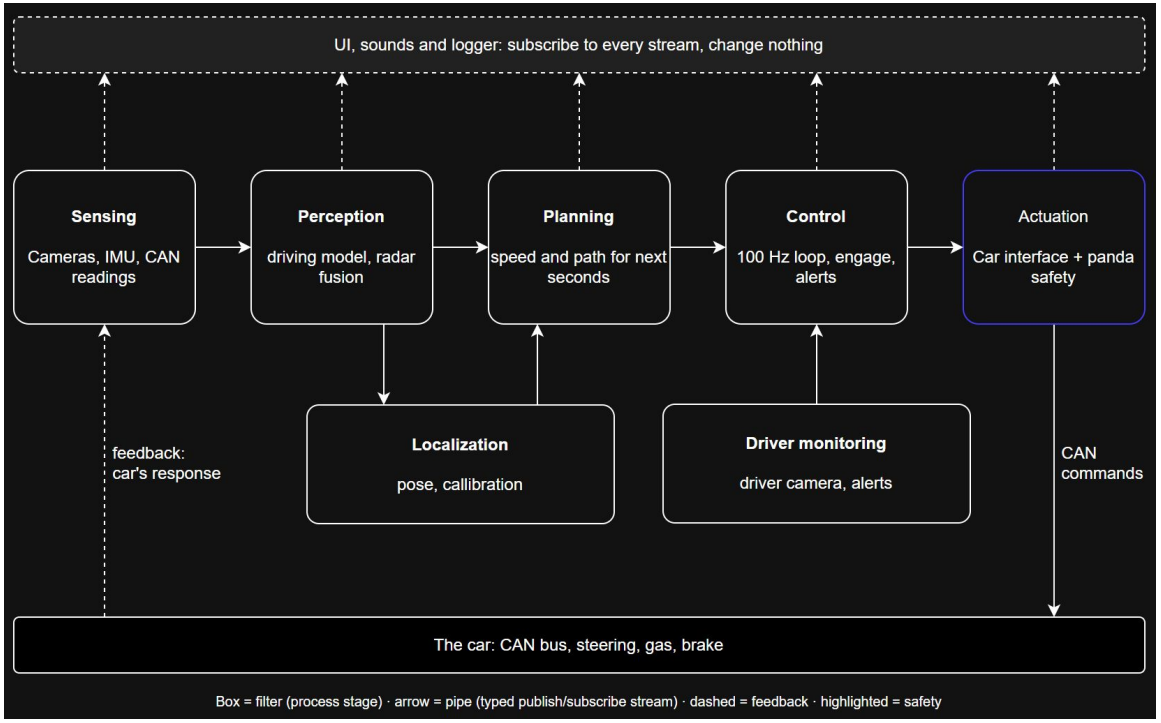


What OpenPilot Does, and How It’s Split Up

A Level 2 driver assist that drives through the car’s own CAN (Controller Area Network) system: adaptive cruise, lane centering, lane changing, FCW (Forward Collision Warning)/LDW (Lane Departure Warning), driver monitoring

Microservices?

Many small “services”, each started and stopped on its own



Pipe and filter.

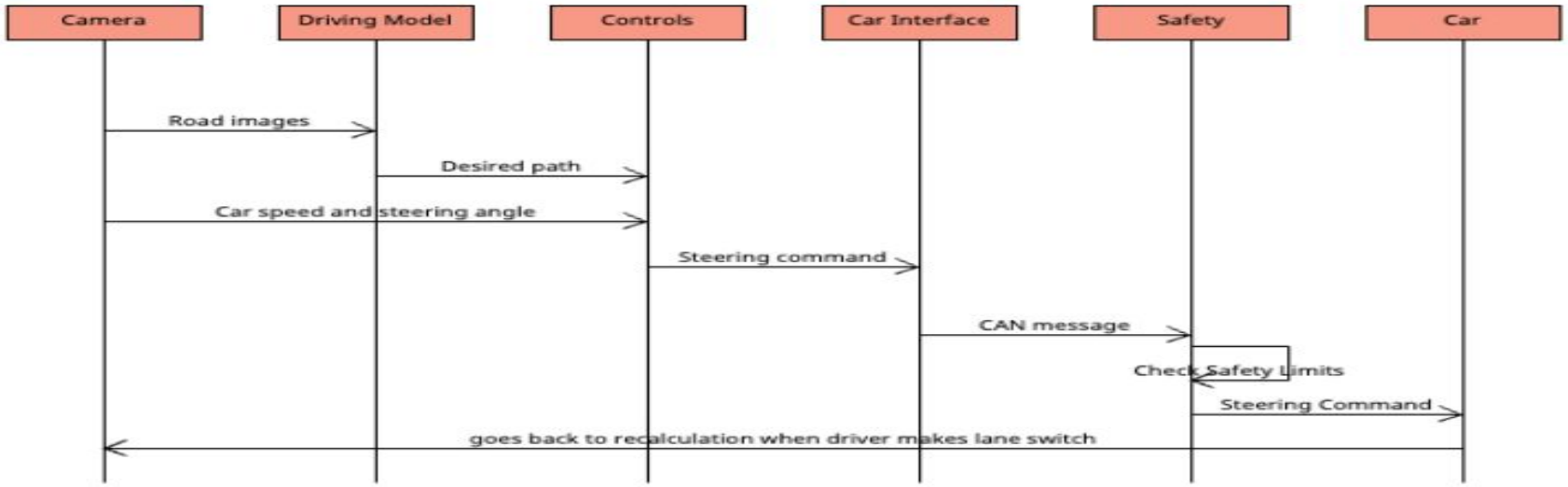
One release on one device

Local message pipes, not network calls

A continuous stream, not request and response

YORK U

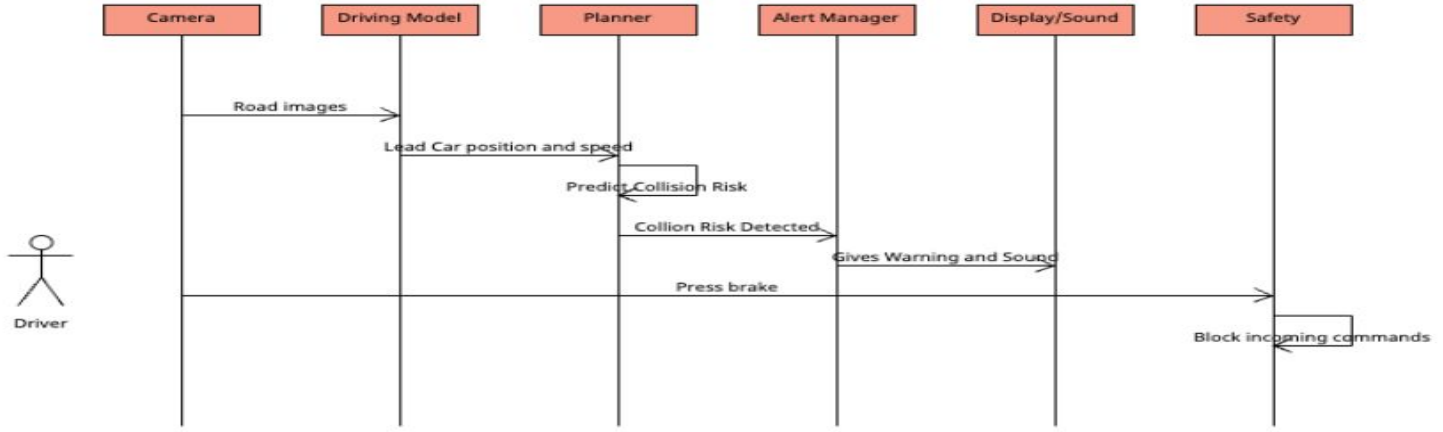
Breaking down 2 main functionalities - ALC & FCW



1. Camera sends road images to the driving model and predicts the path to follow (selfdrive/modeld/modeld.py)
2. Based on the car's current speed and steering angle, controls calculate the steering command (controlsd.py)
3. Car interface turns the command into a CAN message for that specific car (card.py)
4. Do safety check to see if the command is within safe limits before sending it to the car

YORK U

Breaking down 2 main functionalities - ALC & FCW



1. Camera sends road images to the driving model (selfdrive/modeld/modeld.py)
2. The planner looks at the car infront and predict if that will get too close to your car (longitudinal_planner.py)
3. If there's a risk, the alert shows you the warning.
4. When driver brakes, the safety layer blocks the incoming commands and disengages the openpilot.

YORK U

Control and Data Flow in OpenPilot

Data Collection

Collect camera images and vehicle sensor data.

Perception & Planning

Understand the road and determine the vehicle's intended movement.

Control & Interface

Calculate steering and acceleration/braking commands and send them toward the vehicle.

Data flow: Information is processed in stages.

Subsystem communication: Components exchange messages through Cereal.

Feedback: Vehicle-state information helps update control decisions.

Concurrency in OpenPilot

Many processes

OpenPilot runs as many small programs at the same time, started and watched by a manager

Different speeds

Perception runs about 20 times a second. Vehicle control runs about 100 times a second.

Shared messages

Programs share results by passing messages, as a result one slow part doesn't block the others.

Parallelism: independent parts of the system work at the same time rather than waiting in line, so the car can see, plan and steer all at once.

Reliability: if one process fails or crashes, the manager can restart it without shutting down the whole system.

Safety: the panda microcontroller enforces safety limits and checks every command before it reaches the car, so even a software error can't push the car past its safety limits.

How does the architecture impact development responsibilities

Clear separation between each subsystem in function and purpose

Implication?

- Developers can specialize in each area.
  - Keeps each scope small and well defined
  - Hardwares for vehicle control, Machine learning for perception, etc
- Developers can work in parallel and independently
  - Fewer dependencies leading to lower slowdowns in dev tasks
- More coordination between teams
  - Interfaces must be clearly defined and determined beforehand
  - Minor changes can propagate downstream very easily
    - Such deeper emphasis on integration testing and sitewide simulations